FINAL PROJECT REPORT
SOFTWARE ENGINEERING - SPRING 2026

0

Name, Surname	Melisa Gürlük
Student ID	230303007
Name of the Project	FreeLanceOS – Project Management System

1 Introduction

Managing multiple software or engineering projects simultaneously is a significant challenge for freelancers. Tracking budgets across different stages (total revenue, money actually paid, and outstanding receivables) often leads to human error if done manually.

FreeLanceOS is designed to solve this exact problem. It is a lightweight, responsive, and secure project management web application. It allows independent developers to track their financial state in real-time, organize projects by technical categories, receive automatic visual warnings for close deadlines, and log in securely through an encrypted authentication system.

2 System Architecture & Tech Stack

The application is built using a clean, layered architecture to keep the code organized and easy to maintain.

- **Backend:** Python 3 with the Flask framework. Flask handles server-side routing, user sessions, and database transactions cleanly.
- **Database:** SQLite. It provides an embedded, relational, and ACID-compliant database system that stores all data locally in a single file without needing an external database server.
- **Frontend:** HTML5 and a customized dark-themed CSS3. The application avoids heavy frontend frameworks to maintain high rendering speeds and consistent layouts.

3 Database Design & Relationships

The database layer consists of two core tables: `users` and `projects`. They are linked together with a One-to-Many (1:N) relationship using foreign keys, meaning each project belongs exclusively to one specific user.

Table 1: User Authentication Table Structure (`users`)

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for each individual user.
username	TEXT	UNIQUE, NOT NULL	Alphanumeric token used for session login.
password	TEXT	NOT NULL	One-way cryptographically secured hashing string.

To prevent major security risks, raw passwords are never saved into the database. They are processed using the Scrypt algorithm, turning plain text strings into a unique cryptographic signature before saving.

3.1 Use Case Diagram

The following diagram illustrates the functional interactions between the Freelancer (Actor) and the core sub-systems of FreeLanceOS, mapping out authentication limits and project portfolio boundaries.

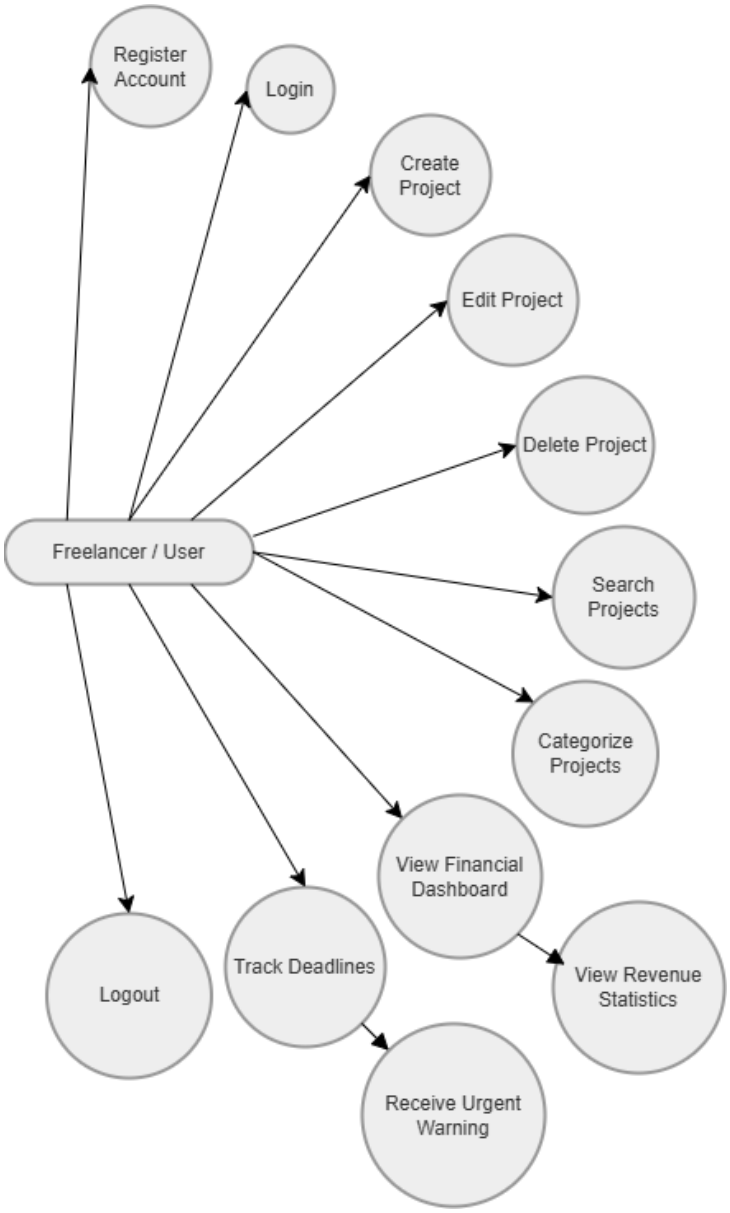

Figure 1: FreeLanceOS General Use Case Diagram Layout.

4 Core Features & User Stories

4.1 Dynamic Financial Analytics Engine

The dashboard calculates the user's financial overview on every page load by parsing active project data arrays:

- **Gross Revenue:** Total sum of all projects combined.

- **Total Earnings:** Sum of budgets where status is explicitly marked as 'Paid'.
- **Outstanding Receivables:** Total sum of budgets where status is still 'Pending'.

4.2 [US4 & US5] Advanced Query Search and Categorization

The system includes an inline search bar utilizing SQL `LIKE` queries to filter rows by client name or project title simultaneously. Additionally, projects are classified using strict categories (e.g., Software & Tech, Engineering & CAD) displayed cleanly in the main layout table to keep entries organized.

4.3 [US6] Automated Proactive Deadline Tracking

To ensure projects are delivered on time, a date-comparison logic runs in the backend. If a project's status is `Pending` and the deadline is less than 3 days away, the system displays a high-contrast `Urgent` badge next to the date without breaking the UI grid alignment.

4.4 [US7] User Authentication and Multi-Criteria Password Policy

A strict security policy is enforced during user registration. Instead of annoying the user with constant error messages after submitting, the specific rules are displayed statically under the form fields:

- Minimum length of 8 characters.
- Must contain at least one uppercase letter (A-Z).
- Must contain at least one lowercase letter (a-z).
- Must contain at least one numerical digit (0-9).

5 Security Infrastructure & Verification

User data protection is handled using the `werkzeug.security` framework. When a user creates an account, the plain text password is fed into a single-way hash function.

During the login process, the password entered by the user is verified against the stored database hash using a constant-time comparison algorithm, protecting the system from side-channel timing attacks.

6 Unit Testing & Verification

To ensure the backend financial calculations always work perfectly and do not break during code refactoring, we implemented automated unit testing using Python's native `unittest` library.

The script `test_finance.py` runs isolated test cases by creating a mock database structure, inserting dummy projects with different budgets and statuses (Paid/Pending), and checking if the functions return the exact expected mathematical values.

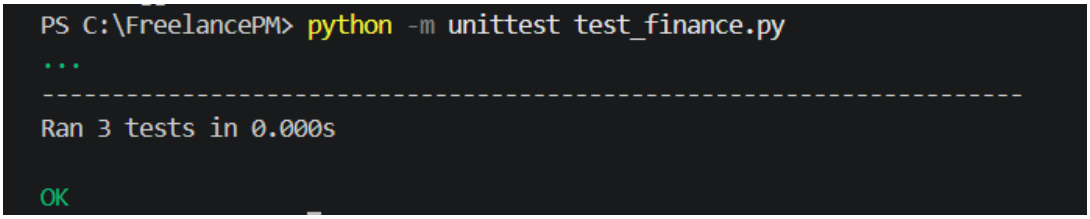

Figure 2: Terminal Output showing all Unit Tests successfully passing (OK).

The automated testing confirms that the core application functions reliably under all circumstances before being deployed to a live environment.

7 Conclusion

FreeLanceOS successfully implements a secure, well-structured, and optimized environment for independent developers. By combining database normalization, secure password hashing, automated deadline notifications, and verified unit testing practices, the project demonstrates essential software engineering principles in a practical and user-oriented manner.

The system provides a lightweight and efficient solution for freelancer project management while maintaining usability, security, and maintainable backend architecture. Overall, the project is suitable for real-world small-scale usage and provides a strong foundation for future improvements such as cloud deployment, REST API integration, and advanced analytics features.